

Practical No.14

MOHAMMAD QASIM

08-11-2025

Objective:

Practical: Import the data set named "Housing" and perform the following:

1) Fit a multiple linear regression model using:

a) `ols()`

b) `OLS()`

c) Generalised Linear model.

2) Fit a multiple linear regression model using Scikit-learn with different Cross-Validation methods.

```
In [1]: import pandas as pd
```

```
In [2]: data=pd.read_csv("Housing_data.csv")
```

```
In [3]: data.head(10)
```

```
Out[3]:
```

	Unnamed: 0	Area	Bedrooms	Bathrooms	Distance_from_CityCenter	Parking	Age	Price
0	0	1460	4	1	8.095208	1	1	135.78
1	1	1894	2	1	3.531862	0	5	120.87
2	2	1730	3	2	11.956830	1	28	123.33
3	3	1695	4	2	6.242570	1	29	133.35
4	4	2238	3	1	10.996239	1	16	144.15
5	5	2769	4	1	22.198726	1	19	174.07
6	6	1066	2	2	8.784281	1	4	111.32
7	7	1838	3	2	3.930111	1	3	144.14
8	8	930	4	2	9.551148	0	17	108.02
9	9	2082	1	1	22.763883	1	17	120.86

1a) Fitting of the model using `ols()`

```
In [4]: import statsmodels.formula.api as smf
```

```
In [5]: model_formula = smf.ols('Price ~ Area + Bedrooms + Bathrooms + Age', data=data).fit()  
print(model_formula.summary())
```

```

=====
OLS Regression Results
=====
Dep. Variable:          Price    R-squared:                0.900
Model:                  OLS      Adj. R-squared:           0.893
Method:                 Least Squares    F-statistic:             123.5
Date:                  Sun, 16 Nov 2025    Prob (F-statistic):       8.61e-27
Time:                  19:56:32    Log-Likelihood:          -232.08
No. Observations:      60      AIC:                     474.2
Df Residuals:          55      BIC:                     484.6
Df Model:               4
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	22.5842	8.209	2.751	0.008	6.133	39.036
Area	0.0459	0.002	19.340	0.000	0.041	0.051
Bedrooms	5.6261	1.292	4.356	0.000	3.037	8.215
Bathrooms	8.4351	3.280	2.572	0.013	1.862	15.008
Age	-0.5981	0.173	-3.467	0.001	-0.944	-0.252

```

=====
Omnibus:                2.398    Durbin-Watson:           1.857
Prob(Omnibus):          0.301    Jarque-Bera (JB):        2.335
Skew:                   -0.435    Prob(JB):                0.311
Kurtosis:               2.578    Cond. No.                1.09e+04
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 1.09e+04. This might indicate that there are strong multicollinearity or other numerical problems.

1b) Fitting of the model using OLS()

```

In [6]: import statsmodels.api as sm
x = data[['Area', 'Bedrooms', 'Bathrooms', 'Age']]
X = sm.add_constant(x) # add intercept
y = data['Price']

```

```

In [7]: model_matrix = sm.OLS(y, X).fit()
print(model_matrix.summary())

```

```

=====
OLS Regression Results
=====
Dep. Variable:          Price    R-squared:                0.900
Model:                  OLS      Adj. R-squared:           0.893
Method:                 Least Squares    F-statistic:             123.5
Date:                  Sun, 16 Nov 2025    Prob (F-statistic):       8.61e-27
Time:                  19:56:33    Log-Likelihood:          -232.08
No. Observations:      60      AIC:                     474.2
Df Residuals:          55      BIC:                     484.6
Df Model:               4
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	22.5842	8.209	2.751	0.008	6.133	39.036
Area	0.0459	0.002	19.340	0.000	0.041	0.051
Bedrooms	5.6261	1.292	4.356	0.000	3.037	8.215
Bathrooms	8.4351	3.280	2.572	0.013	1.862	15.008
Age	-0.5981	0.173	-3.467	0.001	-0.944	-0.252

```

=====
Omnibus:                2.398    Durbin-Watson:           1.857
Prob(Omnibus):          0.301    Jarque-Bera (JB):        2.335
Skew:                   -0.435    Prob(JB):                0.311
Kurtosis:               2.578    Cond. No.                1.09e+04
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 1.09e+04. This might indicate that there are strong multicollinearity or other numerical problems.

1c) Using GLM (Gaussian Family, Identity Link)

There are two approaches:

i) Fitting of GLM using the

```
statsmodels.genmod.generalized_linear_model.GLM
```

ii) Fitting of GLM using the

```
statsmodels.api.GLM.from_formula
```

i)Fitting of GLM using the

```
statsmodels.genmod.generalized_linear_model.GLM
```

```
In [8]: from statsmodels.genmod.generalized_linear_model import GLM
```

```
In [9]: from statsmodels.genmod import families
```

```
In [10]: import warnings
warnings.filterwarnings("ignore", category=FutureWarning)
glm_model = GLM(y, X, family=families.Gaussian(sm.families.links.identity()))
glm_result = glm_model.fit()
print(glm_result.summary())
```

```
Generalized Linear Model Regression Results
=====
Dep. Variable:          Price    No. Observations:          60
Model:                  GLM      Df Residuals:              55
Model Family:           Gaussian Df Model:                4
Link Function:           identity Scale:                  146.24
Method:                  IRLS    Log-Likelihood:         -232.08
Date:                    Sun, 16 Nov 2025 Deviance:              8043.4
Time:                    19:56:35 Pearson chi2:           8.04e+03
No. Iterations:          3       Pseudo R-squ. (CS):      0.9997
Covariance Type:         nonrobust
=====
               coef    std err          z      P>|z|      [0.025      0.975]
-----
const          22.5842     8.209      2.751     0.006      6.495     38.674
Area            0.0459     0.002    19.340     0.000      0.041      0.051
Bedrooms        5.6261     1.292     4.356     0.000      3.094      8.158
Bathrooms       8.4351     3.280     2.572     0.010      2.007     14.864
Age            -0.5981     0.173    -3.467     0.001     -0.936     -0.260
=====
```

ii)Fitting of GLM using the

```
statsmodels.api.GLM.from_formula
```

```
In [11]: import statsmodels.api as sm
```

```
In [12]: MODEL=sm.GLM.from_formula("Price ~ Area + Bedrooms + Bathrooms + Age",family=sm.families.Gaussian(link=sm.families.links.identity()))
```

```
In [13]: print(MODEL.summary())
```

Generalized Linear Model Regression Results						
=====						
Dep. Variable:	Price	No. Observations:	60			
Model:	GLM	Df Residuals:	55			
Model Family:	Gaussian	Df Model:	4			
Link Function:	identity	Scale:	146.24			
Method:	IRLS	Log-Likelihood:	-232.08			
Date:	Sun, 16 Nov 2025	Deviance:	8043.4			
Time:	19:56:37	Pearson chi2:	8.04e+03			
No. Iterations:	3	Pseudo R-squ. (CS):	0.9997			
Covariance Type:	nonrobust					
=====						
	coef	std err	z	P> z	[0.025	0.975]

Intercept	22.5842	8.209	2.751	0.006	6.495	38.674
Area	0.0459	0.002	19.340	0.000	0.041	0.051
Bedrooms	5.6261	1.292	4.356	0.000	3.094	8.158
Bathrooms	8.4351	3.280	2.572	0.010	2.007	14.864
Age	-0.5981	0.173	-3.467	0.001	-0.936	-0.260
=====						

2) Fitting of the model Using Scikit-learn with Various Cross-Validation Methods

```
In [14]: from sklearn.model_selection import train_test_split, KFold, LeaveOneOut, LeavePOut, cross_val_score
```

```
In [15]: from sklearn.linear_model import LinearRegression
```

```
In [16]: from sklearn.metrics import r2_score, mean_squared_error
```

```
In [17]: X = data[['Area', 'Bedrooms', 'Bathrooms', 'Distance_from_CityCenter', 'Parking', 'Age']]
y = data['Price']
```

2a) Hold Out Cross validation (80:20)

```
In [18]: model = LinearRegression()

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
import numpy as np
print("Hold-out R²:", (r2_score(y_test, y_pred)))
print("RMSE:", (np.sqrt(mean_squared_error(y_test, y_pred))))
```

Hold-out R²: 0.9340634022459438
RMSE: 9.83131545022576

2b) K-Fold Cross Validation (k=5)

```
In [19]: kf = KFold(n_splits=5, shuffle=True, random_state=42)

scores = cross_val_score(model, X, y, cv=kf, scoring='r2')
print("K-Fold R² Scores:", np.round(scores, 3))
print("Average R²:", round(scores.mean(), 3))
```

K-Fold R² Scores: [0.934 0.847 0.773 0.92 0.921]
Average R²: 0.879

2c) Leave-One-Out Cross Validation (LOOCV)

```
In [24]: loo = LeaveOneOut()

scores_loo = cross_val_score(model, X, y, cv=loo, scoring='r2')
print("Mean LOOCV R²:", np.round(scores_loo.mean(), 3))
```

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1266: UndefinedMetricWarning: R
^2 score is not well-defined with less than two samples.
  warnings.warn(msg, UndefinedMetricWarning)
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1266: UndefinedMetricWarning: R
^2 score is not well-defined with less than two samples.
  warnings.warn(msg, UndefinedMetricWarning)
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1266: UndefinedMetricWarning: R
^2 score is not well-defined with less than two samples.
  warnings.warn(msg, UndefinedMetricWarning)
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1266: UndefinedMetricWarning: R
^2 score is not well-defined with less than two samples.
  warnings.warn(msg, UndefinedMetricWarning)
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1266: UndefinedMetricWarning: R
^2 score is not well-defined with less than two samples.
  warnings.warn(msg, UndefinedMetricWarning)
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_regression.py:1266: UndefinedMetricWarning: R
^2 score is not well-defined with less than two samples.
  warnings.warn(msg, UndefinedMetricWarning)
```

r^2 can not be used in LOOCV because on a single data point it is undefined

```
In [22]: scores_loo = cross_val_score(model, X, y, cv=loo, scoring='neg_mean_absolute_error')
print(scores_loo)
print("Mean LOOCV R2:", np.round(scores_loo.mean(), 3))
```

```
[-12.60907814 -1.58291929 -0.32417737 -1.68207195 -0.09155501
-12.24321864 -11.45471995 -2.78297281 -23.73854631 -7.10870874
-0.19552849 -13.48552935 -12.02426297 -14.39626932 -3.56959217
-7.21092626 -0.10420887 -10.77030252 -17.22601126 -0.86214916
-1.89320859 -11.41366545 -11.57555571 -1.35941976 -12.43969293
-3.51878316 -0.63405213 -14.05898948 -20.0364747 -14.2993545
-7.65286408 -9.75228563 -4.33997519 -8.48838299 -4.43343308
-1.8879599 -3.05805957 -18.8399714 -8.45445738 -16.42063032
-1.69175077 -2.55873382 -5.26984406 -11.13712331 -9.53347385
-8.74142403 -4.13631246 -14.43614654 -17.75587327 -17.57359896
-9.75595023 -15.66059098 -1.14446258 -19.73856285 -4.09147632
-10.24827876 -14.79147168 -6.55727693 -29.21203648 -11.61615825]
Mean LOOCV R2: -9.061
```

2d) Leave P-Out Cross Validation

```
In [23]: lpo = LeavePOut(p=2)
scores_lpo = cross_val_score(model, X, y, cv=lpo, scoring='r2')
print(scores_lpo)
print("Mean Leave-2-Out R2:", round(scores_lpo.mean(), 3))
```

```
[ -0.42706474 -1.05127333 -54.02845085 ... -0.96312001  0.94386503
 0.83236684]
Mean Leave-2-Out R2: -4390.138
```

Interpretation:

1) Fitted the multiple linear regression model using:

a) ols():

The model explains 90% of the variation in house prices ($R^2 = 0.90$), indicating excellent fit. All predictors are statistically significant ($p < 0.05$). and model's p-value is $8.6e-27$ which indicate that the model is significant and $\hat{\beta}_0 = 22.582$, $\hat{\beta}_1 = 0.0459$, $\hat{\beta}_2 = 5.6261$, $\hat{\beta}_3 = -0.5981$, and all $\hat{\beta}$ has p-value less than 0.05 that means they are significant, AIC value is 474.2 Area, Bedrooms, and Bathrooms positively affect price, with Area having the strongest impact. Age has a negative effect, meaning older houses tend to be cheaper. The intercept and all coefficients are meaningful. Residual diagnostics suggest the errors are approximately normal, and the Durbin-Watson value (~ 1.86) indicates no major autocorrelation issues. Overall, this is a strong and reliable regression model.

b) OLS():

The model explains 90% of the variation in house prices ($R^2 = 0.90$), indicating excellent fit. All predictors are statistically significant ($p < 0.05$). The model's p-value is $8.6e-27$, which indicates that the model is highly significant. $\hat{\beta}_0 = 22.5842$, $\hat{\beta}_1 = 0.0459$, $\hat{\beta}_2 = 5.6261$, $\hat{\beta}_3 = 8.4351$, $\hat{\beta}_4 = -0.5981$, and all $\hat{\beta}$ have p-values less than

0.05, meaning they are significant. The AIC value of the model is 474.2. Area, Bedrooms, and Bathrooms positively affect price, with Area having the strongest impact. Age has a negative effect, meaning older houses tend to be cheaper. Residual diagnostics suggest the errors are approximately normal, and the Durbin-Watson value (~1.86) indicates no major autocorrelation issues. Overall, this is a strong and reliable regression model.

c) Generalised Linear model.

The model explains a very high proportion of variation in house prices (Gaussian GLM with strong fit). All predictors are statistically significant ($p < 0.05$). The model's p-value (from all coefficients) shows strong significance, indicating the model is reliable. $\hat{\beta}_0 = 22.5842$, $\hat{\beta}_1 = 0.0459$, $\hat{\beta}_2 = 5.6261$, $\hat{\beta}_3 = 8.4351$, $\hat{\beta}_4 = -0.5981$, and all $\hat{\beta}$ have p-values less than 0.05, meaning they are significant. The Log-Likelihood is -232.08 and Deviance is 8043.4. Area, Bedrooms, and Bathrooms increase price, with Area having the strongest positive effect. Age negatively affects price, indicating older houses are cheaper. Overall, the GLM provides a strong and consistent model for predicting house prices.

2) Fit a multiple linear regression model using Scikit-learn with different Cross-Validation methods.

2a) Hold Out Cross validation (80:20):

Hold-out R^2 : 0.9340634022459438

RMSE: 9.83131545022576

The hold-out cross-validation results show strong predictive performance. The hold-out R^2 is 0.9341, meaning the model explains about 93% of the variation in unseen test data. The RMSE is 9.83, indicating low prediction error and good accuracy. Overall, the model generalizes well and performs reliably on new data.

2b) K-Fold Cross Validation (k=5)

K-Fold R^2 Scores: [0.934 0.847 0.773 0.92 0.921]

Average R^2 : 0.879

The 5-Fold cross-validation results indicate strong and consistent model performance. The R^2 scores across the folds are [0.934, 0.847, 0.773, 0.920, 0.921], showing stable predictive ability. The average R^2 is 0.879, meaning the model explains about 88% of the variance on average across folds. Overall, the model demonstrates good generalization performance with low variability in accuracy.

2c) Leave-One-Out Cross Validation (LOOCV)

The Leave-One-Out Cross Validation (LOOCV) could not provide a valid R^2 score. An UndefinedMetricWarning was generated because R^2 is not well-defined when the test set contains only one sample, which occurs in LOOCV. This means the model cannot compute R^2 for each individual left-out observation. Overall, LOOCV is not suitable in this case for evaluating R^2 due to the one-sample limitation.

2d) Leave P-Out Cross Validation:

[-0.42706474 -1.05127333 -54.02845085 ... -0.96312001 0.94386503 0.83236684]

Mean Leave-2-Out R^2 : -4390.138

The Leave-P-Out cross-validation results show highly unstable performance. The individual R^2 scores vary widely, ranging from small negative values to very large negative values, indicating poor predictive ability for many subsets. The mean Leave-2-Out R^2 is -4390.138, which is extremely low and suggests severe overfitting or model instability when very small subsets are used for training. Overall, Leave-2-Out CV indicates that the model does not generalize well under this validation method.